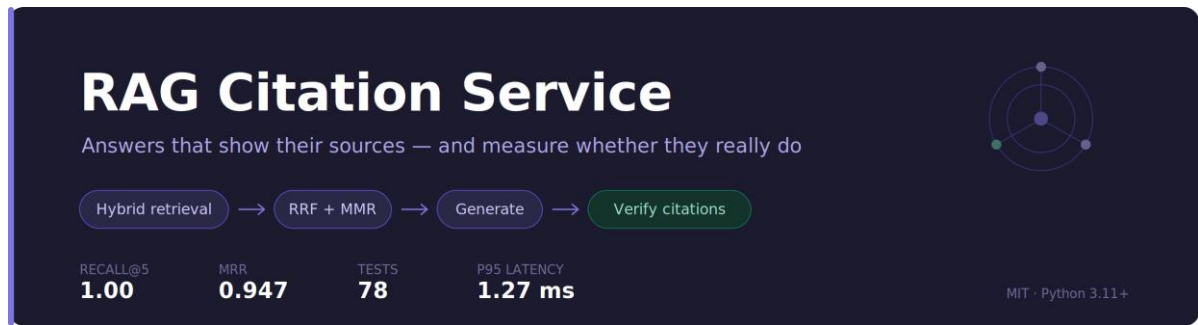




RAG Citation Service

Aditya Gadkari · github.com/adu3010/RAG-Citation-Service

In one paragraph:

This project answers questions from a fixed set of documents and shows the exact passage behind every sentence it writes. It verifies its own citations, scores how well each answer is supported by its evidence, and declines to answer when the documents cannot support one. Its quality is measured automatically against 24 hand-written test questions on every code change, and a change that degrades quality fails the build. It runs offline with no API keys, no paid services and no database.

1. The problem

Ask a general-purpose AI chatbot a question and it will answer confidently whether or not it actually knows. It cannot show you where the answer came from, and it will not tell you when it is guessing.

That is tolerable when you are asking for a dinner recipe. It is not tolerable when a nurse is checking a care pathway, an analyst is checking a financial rule, or a compliance officer is checking a regulation. In those settings a confident wrong answer is worse than no answer, because people act on it and the error propagates.

Two things are missing from the standard approach. First, there is no way to verify the answer without redoing the work yourself, which removes most of the time saving. Second, the system has no concept of not knowing, so it fills gaps with plausible text.

2. What this project does

It answers questions using only a specific set of documents supplied to it, and it shows the exact passage behind every sentence.

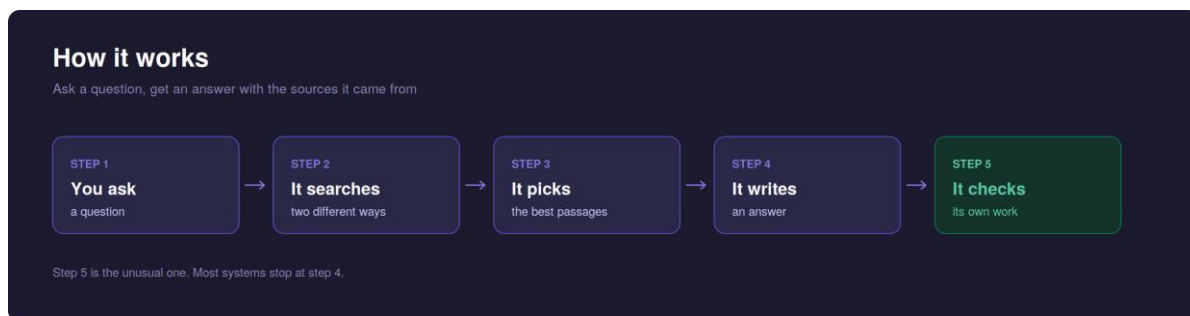
The difference is the one between two colleagues:

"I'm pretty sure the deadline is the 15th."

"The deadline is the 15th — it's on page 4 of the policy, here."

The second colleague is more useful, and the system is built to behave like the second colleague. Every claim comes with a receipt.

3. How it works, step by step



Step 1 — You ask a question

Plain English, phrased as you would ask a person. No special syntax or keywords.

Step 2 — It searches the documents two different ways at once

This is where most systems go wrong, so it is worth understanding. There are two ways to search text and each fails differently.

- **Keyword search** finds exact words. Search “SOC 3544” and it finds documents containing exactly that. But ask about “staff absence” and it will miss a document that says “workforce sickness” — same meaning, different words.
- **Meaning-based search** understands that “staff absence” and “workforce sickness” are the same idea. But it is unreliable with exact codes, identifiers and error strings, which it tends to blur together.

Used alone, either one misses things that matter. This project runs both simultaneously and merges the results, so each covers the other’s blind spot. The technical name is hybrid retrieval.

Step 3 — It picks the best five passages

Two candidate lists come back and have to be combined. The complication is that the two searches score results on completely different scales, so the numbers cannot simply be added together — it would be like adding a temperature in Celsius to a distance in miles.

The solution is to discard the scores entirely and use only the rankings. A passage that both searches place near the top wins. This is a published technique called reciprocal rank fusion, and its practical advantage is that it needs almost no retuning as the document collection grows.

A final tidying step follows. Because passages deliberately overlap at their edges, the raw top five are often five near-copies of the same paragraph. That is actively misleading in a system that displays sources, because five copies of one paragraph look like five independent pieces of evidence. Near-duplicates are therefore dropped in favour of genuinely different passages.

Step 4 — It writes an answer

The answer is written using only those five passages, with a numbered marker after each sentence pointing at the passage it came from.

Step 5 — It checks its own work

This is the step most systems skip entirely, and it is the reason the project exists. It is covered in detail in the next section.

4.The self-checking step

What makes it different

Three guarantees a typical demo does not make

✓ **Shows its sources**

Every sentence points to the exact passage it came from, so you can check it yourself.

✓ **Catches invented sources**

If the answer cites a source that was never retrieved, that citation is removed and logged.

✓ **Says "I don't know"**

When the documents cannot answer the question, it declines instead of guessing.

In healthcare, finance or law, a confident wrong answer is worse than no answer.

Three things happen before an answer is allowed out.

Invented sources are removed

AI models sometimes cite a source that does not exist writing “[7]” when only five passages were supplied. Rendering that as a real citation is arguably worse than having no citations at all, because it manufactures the appearance of evidence. Every citation is checked against the passages that were actually retrieved; any that does not resolve is stripped out and recorded in the logs.

Every answer gets a support score

The system compares each sentence against the passage it cites and scores how well that passage genuinely backs it up. A weakly supported answer is flagged rather than silently returned, so the application using the service can show a warning, route it to a human, or discard it.

It can decline to answer

If the documents do not contain the answer, the correct response is to say so. Systems tuned only to be helpful learn to guess instead, and a plausible guess in a regulated setting is dangerous. This behaviour is explicitly tested: two of the 24 test questions are deliberately unanswerable, and refusing them correctly is scored like any other capability.

5. How we know it works



This is what separates a demonstration from a piece of engineering. There are 24 test questions with known correct answers, written by hand against the document set.

Every time the code changes, all 24 run automatically and the system is scored:

What is measured	Result	In plain terms
Found the right document	100%	The supporting document was always in the top 5
Ranked it first	0.95	It was usually the very first result
Cited the right source	0.80	1 in 5 answers points at the wrong document
Refused correctly	100%	Both unanswerable questions were declined
Wrongly refused	0%	It never refused a question it could answer
Speed	~1 ms	Effectively instant

If a code change makes any of these worse, the build fails and the change is blocked. That is the point of the whole exercise. Without an automatic gate, quality drifts downward invisibly and you discover it only when a user complains.

One further property is unusual: because the default components are deterministic, every quality score above reproduces exactly on any machine. Most systems of this kind cannot reproduce their own numbers between runs.

6. The number that is not 100%

“Cited the right source” is 0.80, not 1.00. That is a real weakness, and it is documented rather than hidden.

The cause is diagnosed. When asked “How many controls are in Annex A of ISO 42001?”, the system treats every word in the question as equally important. A passage matching the common words “many”, “ISO” and “42001” outranks the passage matching the rare, decisive words “Annex” and “controls” — which is the one that actually answers the question.

The fix is known: weight rare words more heavily, using statistics the system already computes for its keyword search. It is the next scheduled change.

This is included deliberately. A project reporting perfect scores across the board tells a reviewer very little, because it usually means the tests were too easy. Knowing precisely where a system fails, and why, is the more valuable signal and the eval harness is what makes that knowledge possible.

7. How it is built

Everything is written from scratch in Python rather than assembled from frameworks, the search algorithm, the ranking, the scoring and the web service. That was a deliberate choice: gluing libraries together teaches you the glue, whereas implementing the algorithms teaches you what they do and where they break.

The service runs anywhere with no API keys, no paid services, no model downloads and no database. Cloning it and running it takes under a minute, offline. Two consequences follow: the automated tests are fast and never fail for network reasons, and the quality scores are exactly reproducible.

Production components are one configuration change away — the meaning-based search and the answer writer can each be swapped for a commercial model without touching any other code.

Numbers at a glance

Item	Value	Note
Source files	40	Application code, tests, corpus, configuration
Automated tests	78	Run on Python 3.11 and 3.12
Test questions	24	Two deliberately unanswerable
External services	0	Runs fully offline
Response time	~1 ms	End-to-end, in process
Licence	MIT	Free to use and modify

8. Where it would be used

Any setting where someone needs an answer from a fixed body of documents and needs to verify it:

- **Clinical and operational policy** — a nurse checking a care pathway against the current guidance
- **Finance** — an analyst checking a treatment against source documentation before signing off
- **Legal and compliance** — searching regulations, where the citation is the entire point
- **Internal knowledge in the company** — a new starter asking about a process and being shown the page it is written on.

The common thread is that the citation matters as much as the answer.

9. Reading the code

If you want to see how a specific part works:

File	Lines	What it does
src/rag/chunking.py	121 lines	Splits documents into passages without cutting sentences in half
src/rag/bm25.py	94 lines	Keyword search, implemented from the published formula
src/rag/embeddings.py	97 lines	Meaning-based search
src/rag/retriever.py	144 lines	Merges the two result lists and removes near-duplicates
src/rag/pipeline.py	286 lines	Verifies citations and scores how well evidence supports the answer
src/rag/api.py	206 lines	The web service
src/rag/eval/runner.py	230 lines	The automatic scoring harness
tests/	78 tests	Behaviour tests, including the concurrency and citation cases

Start with pipeline.py, it is the shortest route to understanding the whole flow.

10. What the tests caught

Worth recording, because it is the clearest evidence the test suite earns its place.

Question words like “what”, “how”, “which” were being treated as meaningful content rather than as grammatical scaffolding. The consequence was that “What is the best pizza in Naples?” matched any passage containing the word “what”, and the system confidently answered a question its documents had nothing to say about, instead of declining.

A test written specifically to check refusal behaviour caught it. The fix required two changes: removing question words from the vocabulary, and introducing a threshold below which the system declines rather than answering on a weak match. Both are now covered by tests, so the behaviour cannot silently regress.